OAuth 2.0

Security Notes — Knowledge Base

1. What is OAuth 2.0?

OAuth 2.0 is an authorization framework that allows a third-party application to obtain limited access to a user's resources on another service, without exposing the user's credentials to that third-party application. Note: OAuth is about authorization (what you can access), not authentication (who you are) — OpenID Connect (OIDC) builds on OAuth 2.0 to add authentication.

2. Key Roles

- Resource Owner — the user who owns the data.
- Client — the application requesting access on the user's behalf.
- Authorization Server — issues access tokens after authenticating the user and obtaining consent.
- Resource Server — hosts the protected resources and validates access tokens (e.g., an API).

3. Common Grant Types (Flows)

Grant Type	Typical Use Case
Authorization Code	Web apps with a backend — most secure, recommended standard flow
Authorization Code + PKCE	Mobile/SPA apps — adds protection against interception
Client Credentials	Machine-to-machine (no user involved)
Refresh Token	Obtaining new access tokens without re-login

4. Authorization Code Flow (Simplified)

- User clicks 'Login with X'; app redirects to the authorization server's login/consent screen.
- User authenticates and approves requested scopes (permissions).
- Authorization server redirects back to the app with a short-lived authorization code.
- App's backend exchanges the code (plus client secret) for an access token (and optionally a refresh token).
- App uses the access token to call the resource server's APIs on the user's behalf.

5. Key Concepts

- Scopes — define exactly what access is being requested (e.g., read:profile, write:calendar).
- Access Token — short-lived credential used to access resources.
- Refresh Token — long-lived credential used to obtain new access tokens without user interaction.

- PKCE (Proof Key for Code Exchange) — protects public clients (mobile/SPA) that can't securely store a client secret.

6. Security Best Practices

- Always use HTTPS for all OAuth endpoints and redirects.
- Validate the 'state' parameter to prevent CSRF attacks during the redirect flow.
- Use PKCE for any client that cannot keep a secret confidential.
- Keep access tokens short-lived and scope them minimally (principle of least privilege).